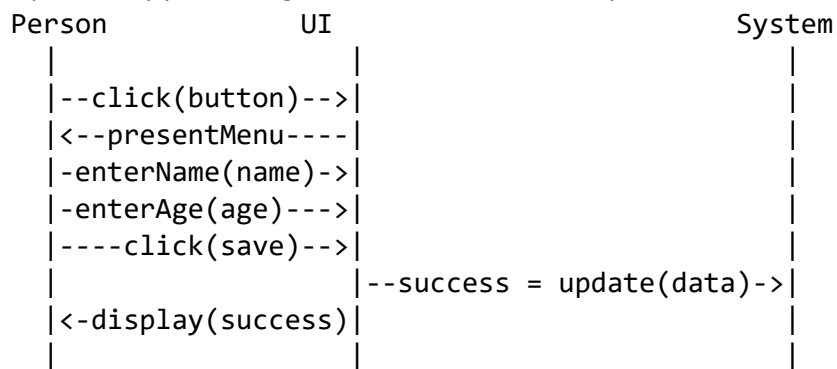


Common SSD errors

- 1) Not creating your Interfaces at the same time. You should create your C++ Interface (an Interface is coded as a class with no attributes and only pure virtual functions) together with your SSD. Each message from the Actor to your system (these are called System Events) should be a function in your Interface class. The function's return type should exactly match the information the system response with. Pay particular attention that the function is not yet implemented. This is consistent with not showing "what" the system does in the sequence diagram, only the information the system response with. These two artifacts (the SSD and the corresponding Interface) should be created and maintained together, typically before any code is written to implement the Interface.
- 2) SSDs should have only two lifelines, the Actor and the System. Sometimes you will see the External System included, but this is rare and for us at this stage only distracts the audience from seeing the interaction between the Actor and the System. Keep your SSDs to only two lifelines.
- 3) Don't mistake the conversation between Actor and System diagrammed in the SSD as interaction the Actor has with the User Interface. If your SSD has actions like "click" or "press" then you haven't captured the correct interaction. Remember the interaction described in the SSD is between the User Interface Layer and the Domain Layer of your system. See above and **don't** draw your SSDs as I'm about to describe, but sometimes it helps the process by pretending there is a third lifeline on your SSD like this:



In this example, the actions and information exchanged between the person and the specific UI implementation are captured on the left. If you were designing a user interface, this view may be interesting – but not here. The information exchanged between the Actor (regardless of the UI that captured it) and the System is captured on the right. This is the SSD content we're looking for. I intentionally chose to show the Actor's request to the System (e.g. update) as a single line containing a function call and the specific information returned from that function call to reinforce the concept that the System returns information and not actions. Unless you are skilled and confident, perhaps using the single line method will help you return information vice perform actions. This leads me to the next point ...

- 4) The System never directs the UI. In other words, the System doesn't request the UI to do something. If you have a verb on the return line from System to Actor or if your returned data looks like a function call, you haven't captured the correct interaction. If you choose to show return lines on your SSD diagram (I chose the other form in the example above) then show the information returned by the corresponding function call only. The pattern should be: Actor makes a request and the System **responds with** information. If your pattern is: Actor makes a request and the System **responds by** doing something, then you haven't captured the correct interaction.
- 5) A scenario is a path through the Use Case. A sequence diagram, include the System Sequence Diagram, represents one scenario. One scenario. That's the key phrase here. A scenario picks a particular, specific path through the Use Case. The Use Case may talk about Creating, Reading, Updating, and Deleting (CRUD) a record of information, but – for this scenario – which one is it? Pick one. Authenticating to the System may success or fail, but – for this scenario – which one is it? Just as your scenario does not have decisions, your Sequence Diagram should not show alternate paths. It is true that once many diagrams are created and as a shortcut several sequences could be captured in a single diagram, we're not there yet. We are still trying to capture a single scenario (sequence). If your diagram has alternative paths shown, you haven't captured **the** sequence

that you will build and demonstrate. Likewise, a scenario should define exactly how many times a set of steps are to be repeated. Ideally if steps are repeated once or twice you would draw them out. If a set of steps is repeated many times, you may use a Loop Frame to capture that, but you must provide an exact number of times the loop is executed in the Guard clause. For this scenario, are you doing it 3 times? 5 times? For this scenario, which is it? Pick one.